

# ChatGPT - SnakeGuard AI

Trong project này tôi sẽ thực hiện một project để ứng tuyển ai engineer. Hiện tại tôi chỉ có ý tưởng như sau:

- Một hệ thống phát hiện phân loại rắn dựa vào ảnh chụp (có thể vừa kết hợp YOLO + Classification)
- Từ loại rắn đó đưa ra thông tin cho người dùng thông qua RAG chatbot, và cách sơ cứu
- Nếu triển khai được app trên điện thoại thì càng tốt, hoặc ít nhất phải có giao diện streamlit
- Thêm một tính năng: người dùng mô tả đặc điểm -> đưa ra ảnh k loại rắn
- Tính năng nâng cao (chỉ đường đến bệnh viện gần nhất - nếu chỉ được bệnh viện có huyết thanh thì càng tốt)

Ý tưởng sơ khai là như vậy, bạn có thể cụ thể hoá lại được không. Trả lời ngắn gọn

Show moreShow less

Có thể cụ thể hóa thành một project portfolio khá hoàn chỉnh như sau:

## SnakeGuard AI

**Hệ thống nhận diện rắn và hỗ trợ xử lý khi gặp/rắn cắn.**

Luồng chính:

### 1. Snake Detection & Classification

- Input: ảnh người dùng chụp.
- YOLO → phát hiện vị trí con rắn.
- Classifier → xác định loài/top-3 loài có khả năng cao nhất.
- Trả về confidence và cảnh báo nếu model không chắc chắn.

### 2. Snake Knowledge Assistant

- Dùng RAG trên nguồn dữ liệu đáng tin cậy.
- Cung cấp: tên loài, độc tính, phân bố, đặc điểm nhận dạng, mức nguy hiểm.
- Nếu nghi bị cắn → hướng dẫn **sơ cứu an toàn**, đồng thời nhấn mạnh không thay thế tư vấn y tế.

### 3. Describe-to-Snake

- Người dùng nhập: *"rắn màu xanh, đầu tam giác, gặp trên cây..."*
- Semantic/image-text retrieval → trả về **top-K loài + ảnh tham khảo**.
- Có thể dùng CLIP/SigLIP + vector database.

### 4. Emergency Mode

- Lấy vị trí người dùng.

- Tìm bệnh viện gần nhất.
- Nâng cao: xây database bệnh viện có **antivenom/huyết thanh phù hợp** và ưu tiên chỉ đường tới đó.

## 5. Ứng dụng

- MVP: **Streamlit**.
- Sau đó có thể nâng thành FastAPI backend + Flutter/React Native mobile app.

Điểm mạnh khi ứng tuyển **AI Engineer** là project này thể hiện cùng lúc **Computer Vision + multimodal retrieval + RAG/LLM + vector DB + API/backend + deployment**.

Mình sẽ ưu tiên làm theo 3 phase: **MVP nhận diện rắn** → **RAG & Describe-to-Snake** → **Emergency/mobile**, tránh ôm quá nhiều tính năng ngay từ đầu.

Đương nhiên là phải sử dụng Fastapi ngay từ đầu rồi, phải tương tác giữa model và frontend mà

Đúng, nếu mục tiêu là portfolio **AI Engineer** thì nên thiết kế theo hướng service-based ngay từ đầu, không để Streamlit gọi model trực tiếp.

Kiến trúc hợp lý hơn:

### Frontend

- Giai đoạn đầu: Streamlit hoặc React
- Sau này: Flutter/React Native mobile

### FastAPI Backend

- `/predict` → nhận ảnh, gọi YOLO + classifier
- `/chat` → RAG chatbot về loài rắn/sơ cứu
- `/search-by-description` → text → top-K loài rắn
- `/nearby-hospitals` → vị trí → bệnh viện phù hợp
- Quản lý validation, logging, error handling, response schema

### AI layer

- YOLO: detect/crop rắn
- Classification model: phân loại species
- CLIP/SigLIP: retrieval từ mô tả
- Embedding + Vector DB: RAG
- LLM: sinh câu trả lời dựa trên retrieved context

### Data layer

- PostgreSQL: metadata loài rắn, bệnh viện, prediction history
- Qdrant/Chroma/FAISS: vector store
- Object storage/local storage: ảnh

Luồng nhận diện sẽ kiểu:

**Frontend → FastAPI → YOLO → crop → Classifier → species\_id → DB/RAG → response → Frontend**

Như vậy project không chỉ là demo ML mà nhìn giống một **AI product/backend system thực tế**, có API boundary rõ ràng và dễ deploy từng component sau này.

PostgreSQL nên lưu những gì

PostgreSQL nên lưu **dữ liệu có cấu trúc và quan hệ**, không nên dùng để lưu embedding hay file ảnh trực tiếp.

Với SnakeGuard AI, mình sẽ chia tối thiểu như sau:

- **snake\_species**
  - **id**
  - **scientific\_name**
  - **common\_name**
  - **local\_name**
  - **venomous**
  - **danger\_level**
  - **description**
  - **habitat**
  - **distribution**
  - **first\_aid\_note**
- **snake\_images**
  - **id**
  - **species\_id**
  - **image\_url**
  - **source**
  - **is\_verified**

Ảnh thật lưu ở object storage, DB chỉ giữ URL/path.

- **hospitals**
  - **id**
  - **name**
  - **address**
  - **latitude, longitude**
  - **phone**
- **antivenoms**
  - **id**

- name
- description
- hospital\_antivenoms
  - hospital\_id
  - antivenom\_id
  - availability
  - last\_verified\_at
- species\_antivenoms
  - map loài rắn ↔ loại huyết thanh.
- prediction\_history
  - id
  - image\_url
  - predicted\_species\_id
  - confidence
  - top\_k\_predictions dạng JSONB
  - model\_version
  - created\_at
- Nếu có account: users, chat\_sessions, feedback.

Mình đặc biệt khuyên lưu thêm **model\_version + prediction + user feedback**. Sau này bạn có thể nói trong interview rằng hệ thống hỗ trợ **model monitoring / error analysis / data collection for retraining** — điểm này khá có giá trị cho AI Engineer.

Còn **vector embeddings của RAG và Describe-to-Snake** thì để Qdrant/pgvector; nếu muốn giảm số service cho MVP, dùng luôn **PostgreSQL + pgvector** là lựa chọn rất đẹp.

cái tính năng tìm bệnh viện gần nhất á, nên để sau này mở rộng sau. Trước tiên là cần một cái cấu trúc phân chia rõ ràng công việc: có sử dụng fast api, docker, model, CI/CD. Database cần tối thiểu những bảng nào, cấu trúc này phù hợp để sau này mở rộng được sang tính năng bệnh viện

Mình sẽ chốt scope ban đầu theo hướng **production-like nhưng không over-engineer**, và để sẵn đường mở rộng cho hospital feature.

## 1. Kiến trúc tổng thể

Frontend → FastAPI → AI Services → PostgreSQL / Vector DB

Các module chính:

- frontend
  - Streamlit trước, mobile/web app sau
- backend

- FastAPI
- auth nếu cần
- API routing
- business logic
- **model-service**
  - YOLO detection
  - snake classification
  - text-to-snake retrieval
- **rag-service**
  - embedding
  - retrieval
  - LLM response
- **database**
  - PostgreSQL
  - có thể dùng **pgvector** luôn để giảm số service
- **infrastructure**
  - Docker / Docker Compose
  - CI/CD
  - logging
  - model versioning

## 2. Database tối thiểu

Mình nghĩ MVP chỉ cần khoảng **5 bảng cốt lõi**:

| Table                      | Mục đích                           |
|----------------------------|------------------------------------|
| <b>snake_species</b>       | Thông tin chuẩn của từng loài      |
| <b>snake_images</b>        | Ảnh và metadata của từng loài      |
| <b>prediction_logs</b>     | Lưu kết quả inference              |
| <b>knowledge_documents</b> | Metadata tài liệu dùng cho RAG     |
| <b>user_feedback</b>       | Người dùng xác nhận/sửa prediction |

**snake\_species** nên là bảng trung tâm, ví dụ:

```
snake_species
-----
id
scientific_name
common_name
local_name
venomous
danger_level
description
habitat
distribution
```

```
created_at
updated_at
```

#### prediction\_logs:

```
id
image_url
predicted_species_id
confidence
top_k_predictions JSONB
model_name
model_version
lat
lng
created_at
```

Mình **đã để sẵn** `lat`/`lng` trong prediction log. Sau này khi thêm Emergency/Hospital feature, bạn có thể biết prediction xảy ra ở đâu mà không phải redesign toàn bộ hệ thống.

### 3. Chuẩn bị sẵn cho hospital nhưng chưa implement

Sau này chỉ cần thêm:

```
hospitals
hospital_antivenoms
antivenoms
species_antivenoms
```

Quan hệ lúc đó sẽ thành:

`snake_species` → `species_antivenoms` → `antivenoms` → `hospital_antivenoms` → `hospitals`

Tức là schema hiện tại **không phụ thuộc bệnh viện**, nhưng khi mở rộng vẫn nổi tự nhiên qua `snake_species.id`.

### 4. Chia công việc theo phase

#### Phase 1 — Foundation

- Git repo structure
- PostgreSQL schema
- FastAPI skeleton
- Docker Compose
- CI: lint + test + build

#### Phase 2 — Computer Vision

- dataset preparation
- YOLO training
- classifier training
- inference pipeline:

`image` → `YOLO` → `crop` → `classifier` → `top-K`

- expose `/api/v1/predict`

### Phase 3 — Knowledge/RAG

- collect verified snake documents
- chunk + embedding
- pgvector
- `/api/v1/chat`
- prediction → species context → chatbot

### Phase 4 — Describe-to-Snake

- CLIP/SigLIP embedding
- text description → top-K species/images
- `/api/v1/search`

### Phase 5 — Production

- tests
- CI/CD
- Docker image
- deploy FastAPI + PostgreSQL
- monitoring/logging

### Phase 6 — Future

- geolocation
- hospital search
- antivenom availability
- mobile app

Điểm mình sẽ nhấn mạnh trong CV/project report là kiến trúc:

**ML training tách khỏi inference, inference tách khỏi API, API tách khỏi database/frontend.**

Như vậy project thể hiện đúng chất **AI Engineer**, chứ không chỉ là notebook train model + Streamlit demo.

Xây dựng một cái plan để làm project này trong 10 ngày (chưa cài đặt bệnh viện đâu), Mỗi ngày cần có cách step cụ thể

Mình đề xuất plan 10 ngày theo kiểu **mỗi ngày đều có deliverable rõ ràng**, ưu tiên hoàn thành một MVP production-like trước, chưa làm hospital.

### Ngày 1 — Chốt scope + dựng skeleton

- Chốt feature MVP:

- upload ảnh → detect + classify
- RAG hỏi thông tin loài rắn
- mô tả đặc điểm → top-K loài
- Tạo repo structure: **backend/**, **frontend/**, **models/**, **ml/**, **tests/**, **infra/**
- Dựng FastAPI skeleton
- Dựng PostgreSQL + Docker Compose
- Tạo **.env**, config, logging cơ bản
- Thiết kế schema DB ban đầu

**Output:** FastAPI + PostgreSQL chạy được bằng `docker compose up`.

## Ngày 2 — Data pipeline + database

- Chuẩn hóa taxonomy các loài rắn
- Chuẩn bị dataset detection/classification
- Viết script split train/val/test
- Tạo migration/schema:
  - **snake\_species**
  - **snake\_images**
  - **prediction\_logs**
  - **knowledge\_documents**
  - **user\_feedback**
- Seed dữ liệu species ban đầu
- Viết SQLAlchemy models + repository layer

**Output:** database usable + dataset có cấu trúc rõ.

## Ngày 3 — Train YOLO detection

- Annotate/kiểm tra bounding boxes
- Train baseline YOLO
- Đánh giá:
  - Precision
  - Recall
  - mAP
- Export model artifact
- Viết inference function:

**image** → **bounding boxes** → **snake crops**

**Output:** YOLO detect được rắn ổn định trên test images.



## Ngày 4 — Train Classification model

- Chuẩn bị cropped snake dataset
- Train baseline classifier, ví dụ EfficientNet/ConvNeXt
- Data augmentation
- Evaluate:
  - accuracy
  - top-3 accuracy
  - confusion matrix
- Export model
- Ghép pipeline:

**image** → **YOLO** → **crop** → **classifier** → **Top-K species**

**Output:** end-to-end CV pipeline hoạt động local.

## Ngày 5 — Đưa model vào FastAPI

- Tạo model loader singleton/lifespan
- Endpoint:
  - **POST /api/v1/predict**
- Response gồm:
  - detected snake
  - species
  - confidence
  - top-K
  - metadata species
- Lưu prediction vào **prediction\_logs**
- Validate image input
- Error handling
- Viết unit/API tests cơ bản

**Output:** gửi ảnh qua HTTP và nhận kết quả inference hoàn chỉnh.

## Ngày 6 — Xây RAG

- Thu thập tài liệu đáng tin cậy về:
  - đặc điểm
  - độc tính
  - môi trường sống
  - phân bố

- sơ cứu
- Làm ingestion pipeline:
  - clean
  - chunk
  - embedding
- Dùng PostgreSQL + **pgvector**
- Implement retrieval
- Endpoint:
  - **POST /api/v1/chat**
- Cho phép truyền **species\_id** từ prediction sang chatbot

**Output:** có thể hỏi *“Loài này có độc không?”*, *“Nếu bị cắn cần làm gì?”*

## Ngày 7 — Describe-to-Snake

- Dùng CLIP/SigLIP hoặc multimodal embedding
- Embed ảnh đại diện của từng species
- Lưu vector + metadata
- Query:

**text description** → **embedding** → **similarity search** → **top-K snakes**

- Endpoint:
  - **POST /api/v1/search-by-description**
- Trả:
  - species
  - similarity
  - ảnh minh họa
  - đặc điểm

**Output:** nhập mô tả và nhận top-K loài phù hợp.

## Ngày 8 — Frontend

Dùng Streamlit để tập trung vào AI/backend thay vì mất thời gian frontend.

Tạo 3 màn hình/tab:

- **Identify Snake**
  - upload ảnh
  - bounding box
  - prediction/top-K
- **Snake Assistant**

- chatbot RAG
- **Find by Description**
  - nhập mô tả
  - hiển thị top-K + ảnh

Frontend **chỉ gọi FastAPI**, tuyệt đối không load model trực tiếp.

**Output:** user có thể sử dụng toàn bộ hệ thống qua UI.

## Ngày 9 — Docker + CI/CD + quality

Container hóa:

- **frontend**
- **backend**
- **postgres**

Docker Compose:

```
Frontend
  ↓ HTTP
FastAPI
  ↓
PostgreSQL + pgvector
```

GitHub Actions:

- lint
- unit tests
- API tests
- Docker build

Thêm:

- Ruff/Black
- Pytest
- health endpoint `/health`
- structured logging

**Output:** clone repo → `.env` → `docker compose up` là chạy.

## Ngày 10 — Polish thành portfolio project

Không thêm feature lớn nữa.

Tập trung:

- benchmark model
- test edge cases

- sửa bugs
- README kiến trúc
- architecture diagram
- API documentation
- screenshots/demo GIF/video
- model metrics
- limitations
- future roadmap:
  - hospital search
  - antivenom DB
  - geolocation
  - mobile app
  - monitoring/retraining

README nên kể được câu chuyện:

**Problem** → **Architecture** → **Data** → **Model** → **Evaluation** → **Backend** → **RAG** → **Deployment** → **Limitations** → **Future Work**

**Mốc quan trọng nhất:** hết **Ngày 5 phải có ảnh** → **FastAPI** → **YOLO** → **classifier** → **response chạy hoàn chỉnh**. Từ ngày 6 trở đi mới thêm AI features khác. Nếu CV pipeline chưa ổn thì nên hy sinh bớt feature RAG/search thay vì có nhiều tính năng nhưng phần lõi chưa tốt.

Nếu làm đúng scope này, sau 10 ngày bạn sẽ có một project khá đầy đủ về **CV + LLM/RAG + FastAPI + PostgreSQL/pgvector + Docker + CI/CD + frontend** thay vì chỉ là một demo model.

Chuyển cái plan này sang dạng text để tôi có thể copy và lưu trữ, sau này để đưa cho các đoạn chat hiểu

## SnakeGuard AI — 10-Day Project Plan

### 1. Project Goal

Xây dựng một hệ thống AI hỗ trợ nhận diện rắn từ ảnh và cung cấp thông tin liên quan đến loài rắn. MVP trong 10 ngày gồm các tính năng chính:

- Nhận diện rắn từ ảnh bằng YOLO + Classification.
- Trả về Top-K loài có khả năng cao nhất.
- RAG chatbot cung cấp thông tin về loài rắn và hướng dẫn sơ cứu.
- Người dùng nhập mô tả đặc điểm để tìm Top-K loài rắn phù hợp.

- Giao diện Streamlit.
- Backend FastAPI.
- PostgreSQL + pgvector.
- Docker / Docker Compose.
- CI/CD cơ bản.
- Logging, testing, model versioning.

Tính năng tìm bệnh viện gần nhất và bệnh viện có huyết thanh KHÔNG nằm trong scope 10 ngày đầu, nhưng kiến trúc cần đủ linh hoạt để mở rộng sau này.

## 2. Architecture

Luồng tổng quát:

Frontend

→ FastAPI Backend

→ AI Services

→ PostgreSQL / pgvector

Các thành phần:

### Frontend

- Streamlit ở giai đoạn MVP.
- Không load model trực tiếp.
- Mọi request đều gọi FastAPI.
- Có thể thay bằng React / Flutter / React Native sau này.

### FastAPI Backend

Các API chính dự kiến:

- POST /api/v1/predict
- POST /api/v1/chat
- POST /api/v1/search-by-description
- GET /api/v1/species/{id}
- GET /health

Backend chịu trách nhiệm:

- Validate input.
- Gọi AI pipeline.

- Query database.
- Logging.
- Error handling.
- Lưu prediction history.
- Trả response chuẩn cho frontend.

## Computer Vision Pipeline

Image

- YOLO Detection
- Crop snake
- Classification Model
- Top-K species

YOLO chịu trách nhiệm tìm vị trí con rắn.

Classifier chịu trách nhiệm xác định loài.

## RAG Pipeline

User question

- embedding
- pgvector retrieval
- relevant documents
- LLM
- grounded answer

Prediction có thể truyền species\_id sang chatbot để RAG ưu tiên context liên quan đến loài vừa nhận diện.

## Describe-to-Snake Pipeline

Text description

- CLIP / SigLIP text embedding
- vector similarity search
- Top-K snake species/images

Ví dụ:

"Rắn màu xanh, đầu tam giác, thường sống trên cây"

→ trả về các loài phù hợp nhất và ảnh tham khảo.

## 3. Database

Sử dụng PostgreSQL.

Có thể sử dụng extension pgvector để tránh phải triển khai thêm một vector database riêng trong MVP.

Các bảng tối thiểu:

## snake\_species

Lưu thông tin chuẩn của từng loài.

Fields:

- id
- scientific\_name
- common\_name
- local\_name
- venomous
- danger\_level
- description
- habitat
- distribution
- created\_at
- updated\_at

Đây là bảng trung tâm của hệ thống.

## snake\_images

Lưu metadata ảnh.

Fields:

- id
- species\_id
- image\_url
- source
- is\_verified
- created\_at

File ảnh thật nên lưu trên filesystem hoặc object storage.

Database chỉ lưu URL/path.

## prediction\_logs

Lưu lịch sử inference.

Fields:

- id
- image\_url
- predicted\_species\_id
- confidence
- top\_k\_predictions JSONB
- model\_name
- model\_version
- latitude nullable
- longitude nullable
- created\_at

latitude và longitude được để nullable nhằm hỗ trợ mở rộng tính năng bệnh viện/geolocation sau này.

## knowledge\_documents

Metadata cho tài liệu RAG.

Fields:

- id
- species\_id nullable
- title
- source
- content
- embedding
- metadata JSONB
- created\_at

Embedding có thể sử dụng pgvector.

## user\_feedback

Lưu feedback để hỗ trợ error analysis/retraining.

Fields:

- id
- prediction\_id



- predicted\_species\_id
- corrected\_species\_id nullable
- is\_correct
- comment
- created\_at

Sau này có thể sử dụng dữ liệu này cho:

- model monitoring
- hard-example mining
- retraining dataset

## 4. Future Hospital Extension

Không implement trong MVP.

Sau này có thể thêm:

- hospitals
- antivenoms
- hospital\_antivenoms
- species\_antivenoms

Quan hệ:

snake\_species  
 → species\_antivenoms  
 → antivenoms  
 → hospital\_antivenoms  
 → hospitals

Kiến trúc hiện tại không phụ thuộc hospital feature nên không cần redesign khi mở rộng.

## 5. Repository Structure

Gợi ý:

snakeguard-ai/  
 backend/  
 app/  
 api/  
 core/  
 db/  
 models/  
 schemas/  
 services/

```
repositories/  
main.py  
  
frontend/  
streamlit_app/  
  
ml/  
detection/  
classification/  
retrieval/  
training/  
evaluation/  
  
models/  
detection/  
classification/  
  
data/  
raw/  
processed/  
  
scripts/  
  
tests/  
unit/  
integration/  
  
infra/  
docker/  
  
docs/  
  
docker-compose.yml  
Dockerfile  
README.md  
.env.example
```

---

## 6. 10-Day Execution Plan

### DAY 1 — Project Foundation

Goal:

Dựng skeleton của toàn bộ hệ thống.

Tasks:

1. Chốt MVP scope.
2. Tạo GitHub repository.
3. Tạo repository structure.
4. Setup Python environment.

5. Khởi tạo FastAPI.
6. Tạo GET /health.
7. Setup PostgreSQL.
8. Setup Docker Compose.
9. Setup environment variables.
10. Setup config management.
11. Setup logging cơ bản.
12. Thiết kế database schema.

Deliverable:

docker compose up

phải chạy được:

- FastAPI
- PostgreSQL

GET /health trả về success.

---

## DAY 2 — Database + Dataset Pipeline

Goal:

Có database usable và dataset có cấu trúc.

Tasks:

1. Chuẩn hóa danh sách snake species.
2. Chuẩn hóa taxonomy.
3. Thu thập/kiểm tra dataset.
4. Tạo train / validation / test split.
5. Tạo database migrations.
6. Tạo bảng:
  - snake\_species
  - snake\_images
  - prediction\_logs
  - knowledge\_documents
  - user\_feedback
7. Implement SQLAlchemy models.
8. Implement Pydantic schemas.
9. Implement repository layer.

10. Seed snake species.

11. Viết dataset preprocessing scripts.

Deliverable:

Database có dữ liệu species.

Dataset sẵn sàng để train.

---

## DAY 3 — YOLO Detection

Goal:

Model có thể phát hiện vị trí rắn trong ảnh.

Tasks:

1. Kiểm tra bounding-box labels.

2. Visualize dataset samples.

3. Setup YOLO training.

4. Train baseline model.

5. Evaluate:

- Precision
- Recall
- mAP50
- mAP50-95

6. Kiểm tra false positives.

7. Kiểm tra false negatives.

8. Export model weights.

9. Viết inference module.

Input:

image

Output:

bounding boxes + confidence + cropped snakes

Deliverable:

YOLO detect được rắn trên test images.

---

## DAY 4 — Snake Classification

Goal:

Xác định species từ crop của YOLO.

#### Tasks:

1. Chuẩn bị classification dataset.
2. Crop/organize ảnh theo species.
3. Data augmentation.
4. Chọn baseline model:
  - EfficientNet
  - ConvNeXt
  - hoặc model tương đương
5. Train model.
6. Evaluate:
  - Top-1 Accuracy
  - Top-3 Accuracy
  - Precision / Recall
  - Confusion Matrix
7. Phân tích các class dễ nhầm.
8. Export model weights.
9. Viết classification inference module.
10. Ghép với YOLO.

Pipeline cuối ngày:

image

→ YOLO

→ crop

→ classifier

→ Top-K species

Deliverable:

Computer Vision pipeline chạy end-to-end local.

---

## DAY 5 — FastAPI Model Serving

Goal:

Expose toàn bộ CV pipeline qua HTTP API.

Tasks:

1. Implement model loader.
2. Load model một lần khi application startup.
3. Implement endpoint:

POST /api/v1/predict

4. Validate:

- file type
- image size
- invalid image

5. Run:

- YOLO
- crop
- classifier

6. Trả về:

- bounding boxes
- predicted species
- confidence
- Top-K species
- species metadata

7. Lưu prediction\_logs.

8. Lưu model version.

9. Error handling.

10. Viết API tests.

Example flow:

Frontend

→ POST /predict

→ FastAPI

→ YOLO

→ Classifier

→ PostgreSQL

→ JSON response

Deliverable:

Có thể gửi ảnh qua HTTP và nhận kết quả nhận diện hoàn chỉnh.

Đây là milestone quan trọng nhất.

Nếu chưa hoàn thành phần này thì không mở rộng feature mới.

---

## DAY 6 — RAG Knowledge Assistant

Goal:

Chatbot trả lời thông tin về rắn dựa trên nguồn dữ liệu đáng tin cậy.

## Tasks:

1. Thu thập tài liệu về:
  - đặc điểm nhận dạng
  - độc tính
  - habitat
  - distribution
  - triệu chứng
  - sơ cứu
2. Lưu source metadata.
3. Clean documents.
4. Chunk documents.
5. Generate embeddings.
6. Setup pgvector.
7. Lưu embeddings.
8. Implement semantic retrieval.
9. Implement RAG pipeline.
10. Implement endpoint:
 

```
POST /api/v1/chat
```
11. Cho phép truyền species\_id.
12. Prompt LLM chỉ trả lời dựa trên context.
13. Có fallback khi không đủ thông tin.
14. Tránh chatbot khẳng định chắc chắn trong tình huống y tế không đủ dữ kiện.

## Deliverable:

### Có thể hỏi:

"Loài này có độc không?"

"Loài này thường sống ở đâu?"

"Nếu bị loài này cắn thì nên sơ cứu thế nào?"

## DAY 7 — Describe-to-Snake

### Goal:

Người dùng mô tả ngoại hình và hệ thống trả về Top-K species.

### Tasks:

1. Setup CLIP hoặc SigLIP.

2. Chuẩn bị reference images cho các species.
3. Generate image embeddings.
4. Lưu embeddings.
5. Implement text embedding.
6. Implement cosine similarity search.
7. Aggregate image results thành species-level score.
8. Implement endpoint:

POST /api/v1/search-by-description

Input example:

"Con rắn màu xanh, thân nhỏ, sống trên cây, đầu hơi tam giác."

Output:

Top-K species:

- species
- similarity score
- reference image
- short description

Deliverable:

Text description → Top-K snake species hoạt động.

## DAY 8 — Streamlit Frontend

Goal:

Người dùng sử dụng được tất cả tính năng từ UI.

Frontend không được import/load model trực tiếp.

Frontend chỉ gọi FastAPI.

Tạo 3 tab/page chính.

### Identify Snake

- Upload image.
- Preview image.
- Call /predict.
- Draw bounding boxes.
- Show Top-K predictions.
- Show confidence.
- Show snake information.



- Có button chuyển sang chatbot với species context.

## Snake Assistant

- Chat interface.
- Gửi message tới /chat.
- Giữ session history.
- Hiển thị source nếu có.

## Find by Description

- Text box.
- Call /search-by-description.
- Hiển thị Top-K species.
- Hiển thị reference images.
- Hiển thị similarity scores.

Deliverable:

User có thể sử dụng toàn bộ hệ thống qua Streamlit.

# DAY 9 — Docker + CI/CD + Testing

Goal:

Project có thể build và deploy giống một ứng dụng thực tế.

Tasks:

1. Dockerize FastAPI.
2. Dockerize Streamlit.
3. Setup PostgreSQL + pgvector container.
4. Hoàn thiện docker-compose.yml.

Architecture:

Streamlit

→ FastAPI

→ PostgreSQL + pgvector

5. Setup GitHub Actions.

Pipeline:

push / pull request

→ lint

→ unit tests

→ API tests

→ Docker build

6. Setup:

- Ruff
- formatting
- Pytest

7. Unit tests cho:

- repositories
- AI services
- utility functions

8. Integration tests cho API.

9. Structured logging.

10. Kiểm tra model loading.

11. Kiểm tra startup time.

12. Kiểm tra invalid input.

Deliverable:

Một người khác có thể:

git clone

sau đó:

docker compose up

và chạy được project.

## DAY 10 — Evaluation + Portfolio Polish

Goal:

Biến project từ demo thành portfolio AI Engineer.

Không thêm feature lớn.

Tasks:

### Model Evaluation

Tổng hợp:

Detection:

- Precision
- Recall
- mAP50

- mAP50-95

Classification:

- Top-1 Accuracy
- Top-3 Accuracy
- Confusion Matrix

Describe-to-Snake:

- qualitative examples
- retrieval success examples

RAG:

- một số test questions
- retrieval relevance
- hallucination/failure examples

## Error Analysis

Phân tích:

- species dễ nhầm
- ảnh tối
- blur
- snake quá nhỏ
- occlusion
- multiple snakes
- background giống snake
- species ngoài training set

## README

README nên có:

1. Problem
2. Motivation
3. Features
4. Architecture
5. Technology Stack
6. Dataset
7. Training
8. Model Evaluation

9. API
10. Database
11. RAG
12. Deployment
13. Screenshots
14. Limitations
15. Future Work

## Portfolio Materials

Chuẩn bị:

- architecture diagram
- screenshots
- example predictions
- confusion matrix
- demo GIF/video
- API Swagger screenshot

## Future Roadmap

Ghi rõ các hướng mở rộng:

- hospital search
- geolocation
- antivenom database
- species ↔ antivenom mapping
- mobile application
- authentication
- cloud object storage
- model monitoring
- data drift detection
- retraining pipeline
- human feedback loop

Deliverable:

Project sẵn sàng đưa vào:

- CV
- GitHub
- portfolio

- AI Engineer interview

## 7. Key Milestones

Milestone 1 — End Day 2

FastAPI + PostgreSQL + dataset foundation hoàn thành.

Milestone 2 — End Day 5

Core pipeline hoàn thành:

Image

→ FastAPI

→ YOLO

→ Classification

→ PostgreSQL

→ Response

Đây là milestone bắt buộc.

Milestone 3 — End Day 7

Ba AI capabilities hoàn thành:

- Image recognition
- RAG chatbot
- Text-to-snake retrieval

Milestone 4 — End Day 9

Toàn bộ hệ thống chạy bằng Docker và có CI.

Milestone 5 — End Day 10

Project đủ chất lượng để đưa vào portfolio.

## 8. Priority Rules

Nếu thiếu thời gian, ưu tiên theo thứ tự:

1. YOLO + Classification hoạt động tốt.
2. FastAPI architecture sạch.
3. Database + logging.
4. Docker.
5. Testing.
6. RAG.
7. Streamlit UI.
8. Describe-to-Snake.

## 9. Advanced polish.

Không hy sinh chất lượng core inference chỉ để có thêm nhiều feature.

Project cần thể hiện rõ:

ML Training

⇒

Model Inference

⇒

Business Logic

⇒

API

⇒

Database

⇒

Frontend

Các layer phải được tách biệt.

---

## 9. Target Technology Stack

Backend:

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic

Database:

- PostgreSQL
- pgvector

Computer Vision:

- YOLO
- PyTorch
- EfficientNet / ConvNeXt

Multimodal Retrieval:

- CLIP / SigLIP

RAG:

- Embedding model

- pgvector
- LLM

Frontend:

- Streamlit

Infrastructure:

- Docker
- Docker Compose

Testing:

- Pytest

Code Quality:

- Ruff

CI/CD:

- GitHub Actions

Version Control:

- Git / GitHub

## 10. Final Project Definition

SnakeGuard AI là một AI system sử dụng Computer Vision, Multimodal Retrieval và RAG để hỗ trợ nhận diện và tìm hiểu về các loài rắn.

Hệ thống có kiến trúc service-based:

Frontend

- FastAPI
- AI Services
- PostgreSQL / pgvector

Core features:

1. Snake Detection.
2. Snake Species Classification.
3. Top-K prediction.
4. RAG Snake Assistant.
5. First-aid information retrieval.
6. Description-to-Snake retrieval.
7. Prediction logging.

8. User feedback.
9. Dockerized deployment.
10. CI/CD pipeline.

Future features:

- Geolocation.
- Nearby hospitals.
- Antivenom availability.
- Mobile application.
- Production monitoring.
- Automated retraining.

Mục tiêu của project không chỉ là train model nhận diện rắn, mà là xây dựng một AI application end-to-end thể hiện các kỹ năng cần thiết của một AI Engineer:

Computer Vision + LLM/RAG + Multimodal AI + API Design + Database + Docker + Testing + CI/CD + Deployment.

ChatGPT can make mistakes. Check important info.